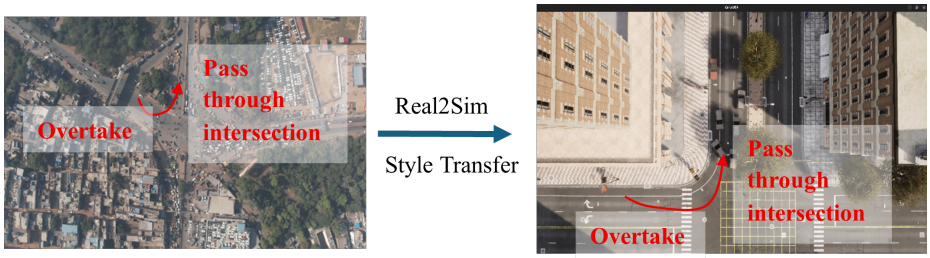
While in London do as the Gwaliorites do: Style Transfer of Trajectories across Traffic Domains in Real-to-Sim

Dasari Raj Vamsi[1 [0009−0000−9637−5007]] and Rahul Kala[1 [0000−0003−0421−5028]]

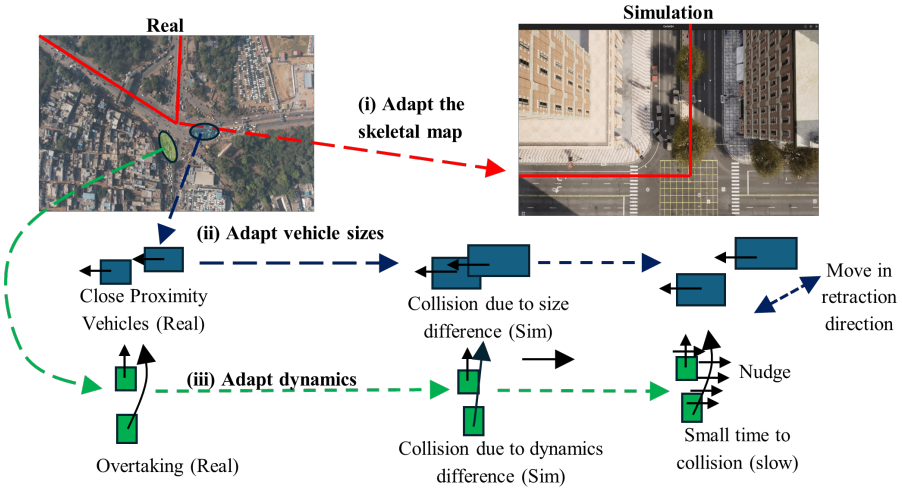
Center for Autonomous Systems,
ABV-Indian Institute of Information Technology and Management Gwalior, India
{imt_2022035,rkala}@iiitm.ac.in

Abstract. Increasing efforts are now being put into creating Real2Sim systems that transform real data into corresponding digital twins, with special focus on driving datasets. Such systems allow testing autonomous driving policies against realistic traffic while retaining the speedup, safety, and synthetic-scenario-generation capabilities of simulation. Our motivation is that autonomous driving research is prevalent in American and European traffic, where rule-breaking and aggressive driving are rare, whereas such traits are common in Indian traffic, where autonomous driving research is limited. The two traffic systems differ in terms of vehicle types, vehicle sizes, vehicle dynamics, and maps. Our problem is to *style transfer* the unorganised and aggressive Indian traffic trajectories into American and European traffic to test the autonomous driving solutions under unsafe and heavy-tail behaviour conditions. The real-world trajectories recorded by a high-altitude drone at an intersection in Gwalior, India, are mapped to a different map available in the CARLA simulator. Simulated vehicles in CARLA are driven using a PID controller over the transformed trajectories. Due to differences in size and dynamics, vehicles can collide. In the initial settings, colliding vehicles are set apart by moving them in the retraction direction. Collisions during execution are handled by making the controller collision-aware using time-to-collision metrics and nudging trajectories rightward for moderate time-to-collision. The simulation is used to create a dataset for Behaviour Cloning that transforms the front RGB monocular camera view into the vehicle's control action using a SegFormer-based vision encoder. The performance is compared against two baselines: a model that blindly follows the ground truth trajectories and the CARLA-built-in autopilot. Evaluation is conducted using behaviour classes such as overtaking, following, lane changes, intersections, and varying traffic density levels. Results show that the learned model produces smaller errors while retaining the overall behaviour-class style.

Keywords: Sim2Real · Autonomous Driving · Behavioral Cloning · Vision-Based Control · Synthetic Scenario Generation



(a) Style Transfer from real to simulation. The behaviour styles 'overtaking' and 'intersection' are transferred to Sim adapting for changes.



(b) Adapting for map, vehicle sizes and dynamics such that the resultant simulation is behaviour preserving and collision free.

Fig. 1: Overall concept

1 Introduction

While testing the autonomous driving (AD) solutions [11] in real life is preferable over the use of simulations, in recent years, there has been a lot of interest in the simulators since simulations can be done across several systems in parallel, simulative testing can be much faster than testing on a real vehicle, there is no risk of danger to human life or damage of the vehicle, and the ability to test against synthetic scenarios including rule-breaking drivers, non-ideal traffic conditions, etc. To make simulations realistic, efforts are made to record real data and transform it into a digital twin that closely mimics the natural counterpart. *Real2Sim* techniques [9] [8] create such digital twins from real data.

For learning AD solutions, a key challenge is generating realistic, scenario-diverse training data that preserves the interaction dynamics among multiple

vehicles. While simulators such as CARLA [7] provide rich environments, naïvely spawning vehicles or relying on built-in traffic managers can lead to unrealistic behaviours or loss of correspondence with real-world traffic patterns. A vehicle trained on such data may perceive real data as out of distribution, in which case its output can be unreliable.

Ego-centric traffic data recording (e.g. Argoverse [4], NuScenes [2]), can capture only limited data from a single vehicle with a limited field of view; therefore, drone datasets are now becoming common (e.g. highD [13], MiTra [5], pNEUMA [1]). However, real-world drone trajectory datasets often provide accurate motion information but lack ego-centric visual observations required for vision-based learning. To address this gap, we build a data generation pipeline that converts real-world traffic trajectories into ego-centric simulation episodes in CARLA.

Imagine testing such AD solutions on a digital twin under common American or European traffic conditions, for which abundant datasets exist. Traffic accidents and safety risks are often attributed to non-ideal driving by the ego vehicle or a neighbouring vehicle, yet such driving behaviours constitute a minuscule part of driving datasets, a phenomenon often called the *heavy tail* problem in AD. The heavy-tail behaviours associated with rule-breaking and aggressive driving are so rare that they cannot be enhanced by augmentation techniques, longer sequence recordings, or curated scenarios.

Imagine traffic in another part of the world, where such conditions are common. We record a natural driving dataset in the Indian city of Gwalior using a drone flying at a large altitude. Lane discipline is not strictly enforced, leading to the overall traffic being *unorganised*. Vehicles range from small motorised vehicles to moderately sized 3-wheeled vehicles, to cars, trucks, and buses. In terms of speeds, there are slow-moving motorbikes and 3-wheeled vehicles, as well as high-speed cars. This encourages constant overtaking from both left and right, while small vehicles can fit between lanes, facilitating overtaking. Traffic is dense, heterogeneous and chaotic. The problem is that there is little work to design AD solutions for such traffic. Currently, very few Indian traffic datasets (like METEOR [3], IDD [17], IDD-CRS [16]) highlight rule-breaking and aggressive driving behaviours.

We therefore intend to introduce the rule-breaking, aggressive, and heterogeneous driving styles of Gwalior into American/European traffic (say, London traffic as an example) to induce heavy-tailed conditions for AD testing. The challenge is that the maps are different across cities, and thus, the raw trajectories of Gwalior cannot be mapped to the roads of London. Furthermore, Gwalior features aggressive motorists and 3-wheeled vehicles that are largely absent in London. We therefore only transfer the *driving style* and not the exact trajectories shown in Fig. 1a. Due to differences in vehicle sizes, shapes, and dynamics, replaying exact trajectories will be collision-prone. Vehicles are spawned selectively based on scenario context, temporal overlap, and spatial proximity, ensuring that interaction-critical agents are present while avoiding unnecessary congestion.

Our problem is to replay the driving styles of Gwalior into a map of London shown in Fig. 1b.

1. We first map the *skeletal map* of Gwalior to London. The Gwalior dataset consists of an intersection where the roads meet non-orthogonally, while it is mapped to a London intersection. The matching road segments are identified, and a transformation is computed, while the transformation of the intersection is interpolated to keep the mapped trajectories homeomorphic and diffeomorphic.
2. We thereafter spawn simulated vehicles using the transformed real coordinates. Due to differences in size, collisions may occur, which are corrected by moving the vehicles in the *collision retraction direction*.
3. Finally, simulated vehicles are made to trace the coordinate-transformed trajectories, which may be collision-prone due to differences in sizes or dynamics. We suppress longitudinal motion in case the time-to-collision is too low. This can make the simulated vehicles cautious and change styles. Therefore, if the time-to-collision decreases, the vehicles are nudged to the right (assuming a to-left driving principle) to closely track the real trajectory.

AD systems must operate reliably under a wide range of traffic conditions, including dense multi-vehicle interactions, lane changes, overtaking maneuvers, and complex intersections. We separately test the AD vehicle's capability to perform such interesting maneuvers to ascertain that the vehicle can handle a range of cases, from simple to complex ones. We note the *behavioural styles* exhibited by the vehicles in different scenarios, such as overtaking, following, lane changes, and intersections. Different traffic densities are also classified as behavioural driving styles. Having scenarios mapped to behavioural styles helps us assess the vehicle's ability for each style separately.

Behaviour Cloning (BC) [12] [15] [14] is widely used for AD. In vision-based BC, a policy learns a direct mapping from raw sensory inputs (RGB images) to continuous control commands. Vehicles are driven using an expert during training, while the same data is used to train a model that autonomously drives a vehicle during testing. We use style-transferred trajectories for BC to generate AD solutions and benchmark the learned solutions. We implement a vision-only BC model using a *SegFormer-based transformer backbone* with a regression head for continuous control prediction. We generate a large-scale ego-centric dataset consisting of synchronised RGB images and continuous control labels (speed and steering). The learned policy is evaluated in CARLA Town10HD_Opt and compared against two baselines: a trajectory-following expert that uses ground-truth trajectories transformed into the new map, and CARLA's built-in autopilot. Performance is analyzed using trajectory error, control smoothness, and collision statistics, reported separately across diverse behaviours and traffic densities. There has been little work in incorporating behaviour styles in BC. In one approach [10] the authors separately train and test for 'stop and move' and 'head-on' collision avoidance behaviours. Similarly, the authors in [6] use Reinforcement Learning for the styles of overtaking, passing and crossing.

In summary, the main contributions of this work are:

- We propose a new problem, *style transfer* of trajectories from one domain to another, within the broader Real2Sim framework.
- We use a retraction-distance-based corrective measure to avoid collisions in a static sense, while also using rightward nudging and time-to-collision-based braking as corrective measures in a dynamic sense.
- We identify and annotate different *driving styles*, including overtaking, following, lane changes, intersections, steering, and traffic density, and separately assess the benchmarks against these styles.
- We design a vision-based behavioural cloning framework using a transformer backbone, evaluated against expert and autopilot baselines using scenario-aware metrics.

2 Style Transfer of Trajectories

This work designs an approach for style transfer in Real2Sim problems. The real data consists of an urban intersection in the city of Gwalior, India. The real vehicle trajectories are captured from a large-altitude drone over an urban intersection in the city of Gwalior, India. The drone is flown at an altitude of 600m, covering a ground area of approximately 475m×275m. The choice of the place is due to active rule-breaking and aggressive driving among the road users. A majority of the drivers are motorbike drivers who are naturally aggressive. Lane discipline is not enforced. The traffic is high density (peaking to 100 vehicles per kilometre) and heterogeneous (consisting of cars, buses, trucks, pedestrians, motor-bikes and 3-wheeled vehicles). Intersection adds diversity, while the large area covers straight road cases as well. The intersection geometry is irregular. The drone video was manually annotated to extract all traffic agents. The annotations recorded the bounding box for each agent along with its ID. The orientation of the agent was taken using a derivative of the trajectories. The trajectories were locally smoothened.

The simulation platform uses the Town10HD_Opt map available in the CARLA simulator [7]. The choice of CARLA simulator was due to the naturalness of the images taken from a virtual camera, enabling realistic and high fidelity simulations.

2.1 Skeletal Correction of Map from Real to Sim

The first problem is that the real map does not match the simulated map. No affine transformation can map the real world map into the simulation map. Therefore, the map in the real world is divided into spatial regions such that similar regions in similar spatial settings are available in simulation. Our scenario consists of an urban intersection with roads intersection at non-right angle in the real world which intersect at right angles in the simulation world. We map each road (region) in real world to the corresponding road (region) in simulation. For each region we compute a separate homography matrix that transforms the real

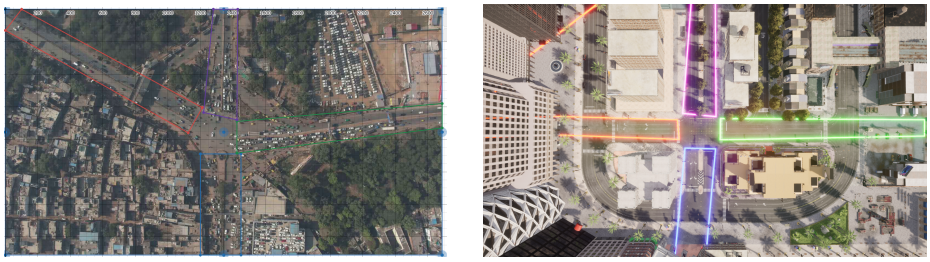


Fig. 2: Skeletal conversion from real to sim. Each coloured region is mapped using an affine transformation and intermediate region is interpolated

world coordinates into the corresponding simulation coordinates. This is shown in Fig. 2.

Direct transitions between region-specific coordinate frames can introduce discontinuities in vehicle motion. The requirement is that the overall transformation is homeomorphic and diffeomorphic to preserve the smoothness of the transformed trajectories. Hence a smooth blending mechanism is applied when a vehicle crosses region boundaries. A continuous blend factor based on a smoothstep function interpolates vehicle states between adjacent regions. This ensures positional and directional continuity and avoids abrupt jumps in the transformed trajectories.

2.2 Local Collision Avoidance During Data Generation

The vehicles in real environment may have much smaller sizes in contrast to those in the simulation setting. The Indian traffic consists of small vehicles which may never be found in the simulated environment modeling a different country's traffic; however, such vehicles are still considered for style transfer due to their aggressive nature of driving.

Difference in sizes causes a collision between vehicles when simulated vehicles are generated at the same position as that of the real vehicles (transformed into the simulation coordinates). Such collisions are handled by moving the colliding vehicles in the retraction direction. The retraction direction is the direction at which two overlapping polygons must be minimally moved so as to eliminate the overlap. Such corrective action ensures that a static frame of real can be converted into a collision-free static frame of simulation. It must be noted that it may not always be possible to correct collisions by using retraction direction, in which case the frame is eliminated.

A PID controller is used to trace the real trajectories of the vehicles by the simulated vehicles (after converting the coordinates from real to sim). The real trajectory is taken as a discrete sequence of points taken at 4 Hz and the next point acts as the immediate goal of the PID controller. The PID controller adapts the linear and angular speed of the vehicle so as to reach the goal.

A scenario is a sequence of frames where vehicles must exhibit smooth transitions realizable using the vehicle's kinematics and dynamics. A collision is possible since the vehicles have different dynamics between real and simulation which is corrected by the PID controller.

Threat detection is based on a time-to-collision (TTC) estimate computed from relative position and velocity. Let $\overrightarrow{p_t}$ be the vector from the ego vehicle to the closest proximity vehicle at time t and let $\overrightarrow{v_t}$ be the current speed vector of the ego vehicle. There is a possible collision in the future if $\overrightarrow{p_t}.\overrightarrow{v_t} > 0$. In such a case the relative position and speed is used to compute the actual TTC.

If TTC is moderately small, a nudge mechanism applies small lateral adjustments when a nearby vehicle is detected as a potential collision threat. Nudging encourages the vehicle to take a little left or right steer. The calculation of the nudging direction is based on the relative direction of the closest proximity vehicle. Let $\overrightarrow{left_t}$ be a vector looking at the left side of the vehicle (produced by rotating the velocity vector $\overrightarrow{v_t}$ by $\pi/2$ along Z-axis). If $\overrightarrow{left_t}.\overrightarrow{p_t} > 0$, the collision threat is to the left and the nudge should be at the right and vice versa. This is shown in Fig. 3.

Nudge is a soft corrective action that eliminates collision by steering while maintaining the vehicle closer to the ground truth trajectories. However, if the TTC is very small, a harder corrective action is applied. Very small TTC triggers vehicle slowing and ultimately stopping. These adjustments operate locally and do not override the underlying trajectory-following behavior.

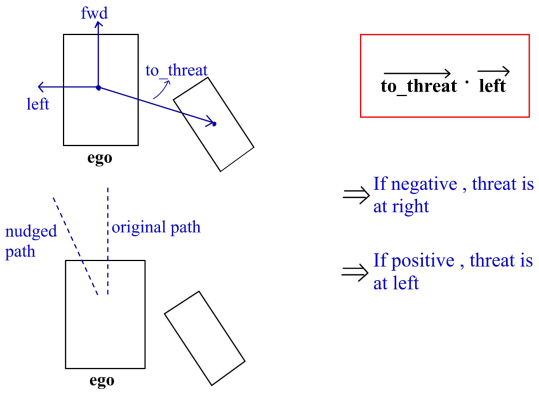


Fig. 3: Calculation of the nudge direction

Vehicle trajectories are extracted from the drone footage and transformed into the simulation coordinate frame over which a collission-aware controller is used to simulate the vehicle. Both the expert-controlled ego vehicle and all other vehicles strictly follow the same process. The original scenarios are dense and hence a sub-sampled number of vehicles are considered, taking the neighbouring

vehicles to preserve behaviours. A minimum of three vehicles are required to form a valid interaction group.

3 Behaviour Cloning

3.1 Problem Formulation

The generated scenarios and dataset are used for end-to-end BC for AD. While it would have been possible to use the CARLA auto-pilot for BC, we aim to learn from the natural driving behaviours particularly amidst aggressive, rule-breaking and lack of lane discipline cases. Therefore, no traffic manager or rule-based AI traffic controller was used at any stage of data generation.

The objective is to learn a continuous control policy that maps perceptual observations and ego-motion state to driving commands, without relying on explicit maps, traffic rules, or planning modules. At each time step t, the ego vehicle observes a multimodal input consisting of: (i) a single monocular RGB image I_t captured from a forward-facing camera, and (ii) a low-dimensional ego-motion state vector $\mathbf{s}_t$. The ego-motion state $\mathbf{s}_t$ includes the vehicle's instantaneous linear speed and angular speed (yaw rate), obtained directly from the simulator.

The learning task is to imitate the expert behavior across diverse behaviour styles. The policy outputs a continuous control vector $\mathbf{u}_t = [v_t, \delta_t]$, where v_t denotes the target longitudinal speed and δ_t denotes the steering command. The action space is continuous and bounded $v_t \in [0, V_{\max}], \quad \delta_t \in [-1, 1]$, where $V_{\max}$ is the maximum allowable speed in the simulation.

Training is performed using supervised learning in an *open-loop* setting, where expert observations are provided regardless of the model's predictions. The objective is to minimize the discrepancy between predicted and expert control commands across all frames $\mathcal{L} = \lambda\, \mathcal{L}_{\text{speed}} + (1 - \lambda)\, \mathcal{L}_{\text{steer}},$, where $\lambda \in [0, 1]$ controls the relative contribution of speed and steering errors. Although training is conducted offline, model evaluation is performed in a *closed-loop* simulation environment, in which a vehicle is loaded with the learned model, performs AD. The vehicle takes live input from the simulation environment and adapts to uncertainties.

A top-down bird's-eye view (BEV) is recorded for visualization and debugging purposes. This view is not used as model input but provides a global reference for understanding vehicle interactions and expert behavior within the intersection. The ego-centric view for BC and the BEV are shown in Fig. 4.

3.2 Model Architecture

We use the SegFormer [18] model for BC. SegFormer is an encoder–decoder model that takes as input a single front-facing RGB image captured from the ego vehicle. Images are resized and normalised. The encoder follows a transformer-based design and processes the input image into a hierarchy of multi-scale feature representations. Self-attention mechanisms allow the encoder to capture both



Fig. 4: Sensor perspectives used during data collection. Left: front-facing RGB camera view used as model input. Right: top-down bird's-eye view for visualization of vehicle interactions.

local visual cues (e.g., lane markings, nearby vehicles) and broader contextual information (e.g., intersections and road geometry). The decoder aggregates the multi-scale features produced by the encoder and projects them into a compact latent representation. Unlike semantic segmentation tasks, the decoder output is not a dense pixel-wise map, but a global feature vector representing the driving context of the current frame.

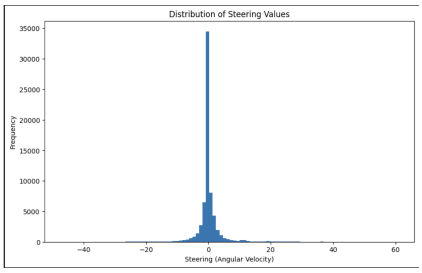
While the SegFormer encoder processes the visual features of the scene, the scalar inputs (linear and angular speed) are processed through a separate multi-layer perceptron. These motion features are then concatenated with the global feature vector derived from the visual decoder before being passed to the final control head. The final latent representation is passed to a fully connected regression head that predicts continuous control commands. The output consists of steering and speed-related control signals, which are applied directly to the vehicle.

3.3 Behaviour Styles

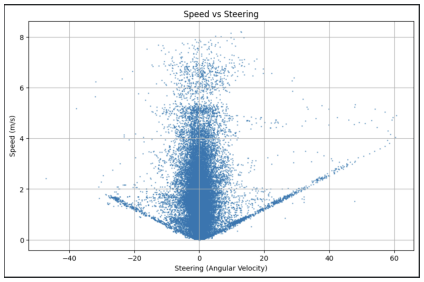
After data generation and evaluation runs are completed, each scenario is annotated with multi-label behaviour style tags for analysis purposes. These tags are not used during training. Behavioral style tags include: `overtaking`, `following`, `lane_change`, and `4-way_intersection`. Traffic density is encoded using labels `dense_0` to `dense_13`, where the numeric suffix denotes the number of vehicles present in the episode. Scenarios may belong to multiple behavior styles and density categories simultaneously. In addition, each scenario is assigned a single turn-type label representing the dominant maneuver: `straight`, `left_turn`, `right_turn`, `slight_left_turn`, `slight_right_turn`, or `unknown`.

3.4 Heavy-Tail Problem

An analysis of the raw steering values reveals a highly imbalanced distribution. As shown in Fig. 5a, the majority of steering commands are concentrated near zero, corresponding to straight or near-straight driving. Large-magnitude steering values occur less frequently and are primarily associated with turning maneuvers at intersections and during lane changes.



(a) Distribution of raw steering values across the dataset. The strong concentration around zero indicates a dominance of straight-driving behavior.



(b) Scatter plot showing the relationship between vehicle speed and steering command. Large steering magnitudes are primarily observed at lower speeds, while straight driving dominates at higher speeds.

Fig. 5: Heavy-Tail problem of training

The relationship between speed and steering magnitude is illustrated in Fig. 5b. Higher steering magnitudes predominantly occur at lower speeds, reflecting cautious turning behavior in dense and unstructured traffic scenarios. Conversely, higher speeds are mostly observed during near-zero steering, corresponding to straight road segments.

To assess potential dataset bias, the training split is analyzed with respect to turn-type labels. Episodes are categorized into straight, left turn, right turn, slight left turn, and slight right turn classes. This analysis confirms the dominance of straight-driving segments while retaining sufficient turning examples to support learning of non-trivial steering behaviors.

4 Experimental Setup and Results

All evaluations are conducted using CARLA v0.9.15 on the *Town10HD_Opt* map. This map provides a high-fidelity urban environment with complex road topology and is well suited for replicating the non-orthogonal intersection geometry observed in the real-world Gwalior drone dataset. The simulator is operated in synchronous mode with a fixed time-step of 0.05 seconds (20 FPS). The training was performed on cloud-based high-performance hardware, the closed-loop

evaluation and simulation are conducted locally on an NVIDIA T1000 GPU (8 GB VRAM).

Optimization is performed using the AdamW optimizer. The model is trained using an NVIDIA A100 GPU via the Google Colab platform. The high computational capacity and large VRAM of the A100 enabled efficient processing of the 66,630-image dataset with the SegFormer-B0 backbone. Automated Mixed Precision (AMP) was utilized to further accelerate the training process and optimize memory throughput.

The SegFormer encoder is initialized from pretrained weights, while the regression head is trained from scratch. In selected experiments, the encoder is frozen during early training stages to stabilize convergence. Dataset partitioning is performed at the scenario level using an 80/10/10 split for training, validation, and testing. All frames originating from the same interaction episode are assigned to a single split.

Performance is compared against two baseline driving policies:

- **Expert**: A trajectory-following expert that replays real-world vehicle trajectories transformed into the CARLA coordinate frame. Note that differences in sizes and dynamics may mean that the ground truth trajectories are not realizable, attempted to be corrected by the PID controller. The expert trajectories may therefore not have the smallest errors.
- **Autopilot**: CARLA's built-in rule-based autopilot, representing a classical heuristic-driven baseline designed for structured lane-based environments.

The performance of the system is assessed using the metrics of Average Displacement Error (ADE) and Final Displacement Error (FDE) that assess the closeness to the ground truth trajectory. Jerk RMS and standard deviation of the yaw rate are used to measure passenger comfort. Collision rate, minimum TTC and completion rate are used to assess safety of the vehicle. Unless stated otherwise, positional metrics are reported in meters, yaw-related metrics in degrees, and jerk-related metrics in m/s^3.

Table 1 reports numerical performance, while Fig. 6 visualizes the same comparisons. The results for different types of turn are shown in Table 2 and Fig. 7. A consistent observation across multiple scenarios is that the proposed model achieves a lower FDE than the expert replayer, particularly in overtaking and following scenarios. The expert policy strictly replays predefined trajectories derived from real-world drone annotations which are subject to real-to-simulation mapping inaccuracies, particularly due to differences in vehicle sizes and dynamics. When executed in a closed-loop simulator, these errors accumulate over time, leading to larger terminal deviations.

In contrast, the proposed model bases its control decisions on the current visual scene and ego-motion state. This enables limited self-correction during execution, allowing the vehicle to remain aligned with the drivable space even when the underlying expert trajectory drifts. As a result, the learned policy can reach a final position closer to the intended path despite being trained purely via imitation learning.

Table 1: Comparative results

Metric	Proposed	Expert	Autopilot	Proposed	Expert	Autopilot
	Overtaking			**Following**		
FDE $(m)\downarrow$	**5.41**	20.40	21.20	**1.07**	33.34	11.16
ADE $(m)\downarrow$	2.50	**1.64**	4.91	**0.43**	2.10	3.40
JerkRMS $(m/s^3)\downarrow$	**80.17**	82.71	90.09	82.16	**57.96**	160.37
YawStd $(^\circ/s)\downarrow$	**1.34**	5.21	13.90	**1.29**	2.18	20.20
Collisions $\downarrow$	0.24	0.23	**0.21**	0.33	**0.08**	0.75
	Lane Change			**4-way intersection**		
FDE $(m)\downarrow$	**8.64**	31.77	23.66	**5.49**	20.60	16.13
ADE $(m)\downarrow$	**3.52**	3.84	6.44	**2.34**	**2.34**	3.75
JerkRMS $(m/s^3)\downarrow$	**92.37**	106.25	138.08	**88.12**	91.71	118.19
YawStd $(^\circ/s)\downarrow$	**1.51**	8.77	22.54	**1.12**	4.44	12.33
Collisions $\downarrow$	**0.14**	0.47	0.45	0.30	**0.24**	0.32
	Dense scenarios			**Sparse scenarios**		
FDE $(m)\downarrow$	**1.53**	16.49	18.90	**5.27**	17.89	15.06
ADE $(m)\downarrow$	**0.44**	0.80	0.51	**2.08**	**2.08**	2.87
JerkRMS $(m/s^3)\downarrow$	76.65	**47.38**	61.92	**97.47**	99.63	124.76
YawStd $(^\circ/s)\downarrow$	**0.72**	2.29	6.17	**1.13**	3.25	10.53
Collisions $\downarrow$	0.24	**0.14**	0.17	0.37	**0.28**	0.45

Table 2: Comparative results

Metric	Proposed	Expert	Autopilot	Proposed	Expert	Autopilot
	Left Turn			**Right Turn**		
FDE $(m)\downarrow$	**2.84**	13.96	15.03	**3.36**	12.67	12.86
ADE $(m)\downarrow$	**1.24**	3.47	5.32	**2.03**	3.17	5.41
JerkRMS $(m/s^3)\downarrow$	**72.27**	101.92	112.19	100.74	**85.25**	190.43
YawStd $(^\circ/s)\downarrow$	**0.88**	7.60	15.18	**1.54**	11.30	24.48
Collisions $\downarrow$	0.33	**0.27**	**0.27**	0.60	**0.50**	0.58
	Slight Left Turn			**Slight Right Turn**		
FDE $(m)\downarrow$	**3.66**	8.68	9.70	**9.59**	22.33	10.29
ADE $(m)\downarrow$	**1.82**	3.25	3.09	5.08	**1.98**	3.28
JerkRMS $(m/s^3)\downarrow$	104.59	**98.42**	136.61	**71.86**	97.86	109.22
YawStd $(^\circ/s)\downarrow$	**1.31**	3.76	13.32	**0.84**	3.31	13.51
Collisions $\downarrow$	0.43	**0.33**	0.35	**0.08**	0.3	0.35
	Straight					
FDE $(m)\downarrow$	**4.62**	18.46	24.24			
ADE $(m)\downarrow$	1.57	**1.45**	1.50			
JerkRMS $(m/s^3)\downarrow$	103.48	**86.69**	87.92			
YawStd $(^\circ/s)\downarrow$	**1.06**	1.62	2.02			
Collisions $\downarrow$	0.35	**0.11**	0.36			

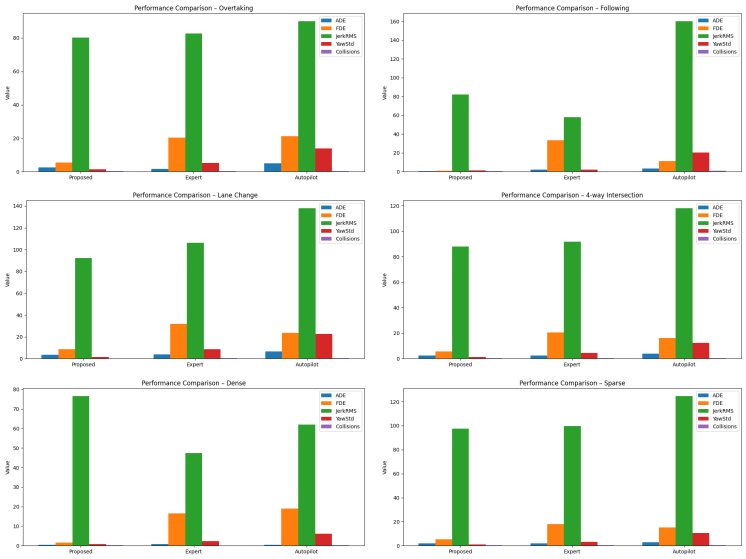


Fig. 6: Scenario-wise performance comparison across Proposed, Expert, and Autopilot policies.

The proposed model consistently exhibits lower Yaw rate standard deviation than both expert and autopilot across the behaviour styles. This behavior is attributed to the multimodal input formulation, where instantaneous linear and angular velocity are fused with visual features extracted by the SegFormer encoder. Providing explicit ego-motion information reduces ambiguity in steering prediction, particularly in non-orthogonal intersection geometries where visual cues alone may be insufficient. The velocity inputs act as a stabilizing prior, suppressing high-frequency steering oscillations commonly observed in purely vision-based behavioral cloning models.

Furthermore, the proposed approach has lower ADE and a lower jerk in most cases, with only a few exceptions, while it is only marginally weaker in the exceptional cases against the harder expert benchmark. In dense traffic conditions, the proposed model exhibits marginally higher JerkRMS compared to the expert baseline, indicating reduced longitudinal smoothness. This behavior arises from the absence of temporal context or predictive planning mechanisms. Operating on instantaneous observations, the model reacts sharply to changes in the visual field, such as sudden deceleration of nearby vehicles. While this reactive behaviour increases longitudinal jitter, it contributes to maintaining safe distances and avoiding collisions. The results highlight an inherent trade-off between smoothness and responsiveness in single-frame imitation learning approaches.

The CARLA AUTOPILOT baseline performs reliably in structured lane-following environments but degrades significantly in the unstructured settings

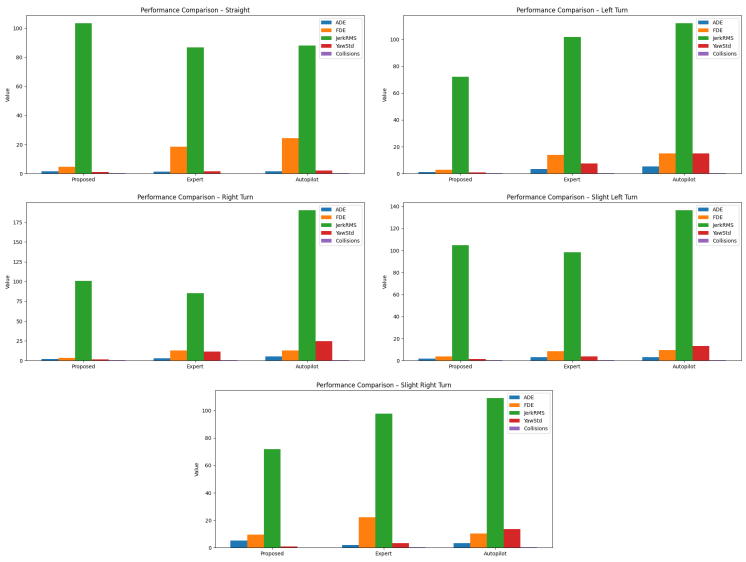


Fig. 7: Turn-type-wise performance comparison across driving policies.

used in this work. Rule-based assumptions regarding lane topology, right-of-way, and intersection structure do not hold in the Gwalior-derived traffic scenarios. In contrast, the proposed model, trained directly on visual and dynamic cues from real-world-inspired data, demonstrates greater adaptability to irregular road layouts and heterogeneous traffic behavior.

5 Conclusions

This paper studied a new problem of style transfer of trajectories from a real domain into a simulation domain. The work reports the initial results of transferring drone footage of an unstructured intersection in Gwalior, India into a CARLA simulation setup. To handle differences in maps, the approach uses a region-based transformations with smooth blending to enable realistic replay of trajectories. A PID controller traces simulated vehicles over simulation coordinate system converted trajectories, while handling collisions using retraction direction, nudging and slowing as per TTC metrics.

This work also presents a vision-based BC framework for AD using the designed style transfer based real2sim framework. Using a SegFormer backbone with multimodal fusion of RGB imagery, linear speed, and angular velocity, the proposed model learns continuous control commands for steering and speed directly from expert demonstrations. The learned trajectories were better than the autopilot of CARLA as well as could beat the ground truth trajectories converted into CARLA coordinates, which failed due to differences in map and

vehicle dynamics. The results are consistent across multiple behaviour styles including traffic densities and turn types.

Despite the encouraging results, several limitations remain in the current framework. Although the source dataset originates from real-world traffic containing heterogeneous agents, the simulation environment used in this work models only four-wheeled passenger vehicles. Two-wheelers such as motorcycles, scooters, and bicycles—common in Indian urban traffic—are not included. The proposed model operates on a single RGB frame combined with instantaneous linear and angular velocity. No temporal stacking, recurrent memory, or future state prediction is used. The perception pipeline relies exclusively on a front-facing RGB camera. Depth information, LiDAR, radar, or multi-camera surround views are not available to the model. This limits awareness of occluded vehicles and peripheral traffic, particularly in wide or irregular intersections.

Additionally, all evaluations are performed within a single simulated town, which may restrict generalization to environments with different visual styles or traffic rules. The framework does not incorporate reinforcement learning fine-tuning, online adaptation, or dataset aggregation techniques such as DAgger. As a result, the model cannot correct systematic errors through experience, particularly in rare or failure-prone scenarios. These limitations shall be worked over in the future research.

References

1. Barmpounakis, E., Geroliminis, N.: On the new era of urban traffic monitoring with massive drone data: The pneuma large-scale field experiment. Transportation research part C: emerging technologies **111**, 50–71 (2020)
2. Caesar, H., Bankiti, V., Lang, A.H., Vora, S., Liong, V.E., Xu, Q., Krishnan, A., Pan, Y., Baldan, G., Beijbom, O.: nuscenes: A multimodal dataset for autonomous driving. In: Proceedings of the IEEE/CVF conference on computer vision and pattern recognition. pp. 11621–11631 (2020)
3. Chandra, R., Wang, X., Mahajan, M., Kala, R., Palugulla, R., Naidu, C., Jain, A., Manocha, D.: Meteor: A dense, heterogeneous, and unstructured traffic dataset with rare behaviors. In: 2023 IEEE International Conference on Robotics and Automation (ICRA). pp. 9169–9175. IEEE (2023)
4. Chang, M.F., Lambert, J., Sangkloy, P., Singh, J., Bak, S., Hartnett, A., Wang, D., Carr, P., Lucey, S., Ramanan, D., et al.: Argoverse: 3d tracking and forecasting with rich maps. In: Proceedings of the IEEE/CVF conference on computer vision and pattern recognition. pp. 8748–8757 (2019)
5. Chaudhari, A.A., Treiber, M., Okhrin, O.: Mitra: A drone-based trajectory data for an all-traffic-state inclusive freeway with ramps. Scientific Data **12**(1), 1174 (2025)
6. Chen, Y.F., Everett, M., Liu, M., How, J.P.: Socially aware motion planning with deep reinforcement learning. In: 2017 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS). pp. 1343–1350. IEEE (2017)
7. Dosovitskiy, A., Ros, G., Codevilla, F., Lopez, A., Koltun, V.: CARLA: An open urban driving simulator. In: Proceedings of the 1st Annual Conference on Robot Learning. pp. 1–16 (2017)

8. Hahn, D., Banzet, P., Bern, J.M., Coros, S.: Real2sim: Visco-elastic parameter estimation from dynamic motion. ACM Transactions on Graphics (TOG) **38**(6), 1–13 (2019)
9. Heiden, E., Millard, D., Sukhatme, G.S.: Real2sim transfer using differentiable physics. In: Workshop on closing the reality gap in sim2real transfer for robotic manipulation. vol. 2 (2019)
10. Jafri, S.H., Kala, R.: End-to-end human inspired learning based system for dynamic obstacle avoidance. Complex & Intelligent Systems **8**(6), 5065–5086 (2022)
11. Kala, R.: On-road intelligent vehicles: Motion planning for intelligent transportation systems. Butterworth-Heinemann (2016)
12. Kala, R.: Autonomous mobile robots: Planning, navigation and simulation. Elsevier (2023)
13. Krajewski, R., Bock, J., Kloeker, L., Eckstein, L.: The highd dataset: A drone dataset of naturalistic vehicle trajectories on german highways for validation of highly automated driving systems. In: 2018 21st international conference on intelligent transportation systems (ITSC). pp. 2118–2125. IEEE (2018)
14. Le Mero, L., Yi, D., Dianati, M., Mouzakitis, A.: A survey on imitation learning techniques for end-to-end autonomous vehicles. IEEE Transactions on Intelligent Transportation Systems **23**(9), 14128–14147 (2022). https://doi.org/10.1109/TITS.2022.3144867
15. Ly, A.O., Akhloufi, M.: Learning to drive by imitation: An overview of deep behavior cloning methods. IEEE Transactions on Intelligent Vehicles **6**(2), 195–209 (2020)
16. Mishra, R.S., Parikh, C., Subramanian, A., Jawahar, C., Sarvadevabhatla, R.K.: Idd-crs: A comprehensive video dataset for critical road scenarios in unstructured environments. In: 2025 IEEE Intelligent Vehicles Symposium (IV). pp. 309–315. IEEE (2025)
17. Varma, G., Subramanian, A., Namboodiri, A., Chandraker, M., Jawahar, C.: Idd: A dataset for exploring problems of autonomous navigation in unconstrained environments. In: 2019 IEEE winter conference on applications of computer vision (WACV). pp. 1743–1751. IEEE (2019)
18. Xie, E., Wang, W., Yu, Z., Anandkumar, A., Alvarez, J.M., Luo, P.: Segformer: Simple and efficient design for semantic segmentation with transformers. Advances in neural information processing systems **34**, 12077–12090 (2021)